Event-Level Fall Localization in Multi-Radar UWB Streams with Boundary-Aware Decoding and Recurrent–Convolutional Fusion

Vu-Nghi Truong*
Dept. of Applied Science
Ho Chi Minh City University of Technology
Ho Chi Minh City, Vietnam
nghi.truongdavid@hcmut.edu.vn

Anh-Khoa Le Nguyen*
Dept. of Artificial Intelligence
University of Economics Ho Chi Minh City
Ho Chi Minh City, Vietnam
khoale.31251025754@st.ueh.edu.vn

Minh-Khoi Le Nguyen*
Dept. of Computer Science and Engineering
Ho Chi Minh City University of Technology
Ho Chi Minh City, Vietnam
khoi.le2550109@hcmut.edu.vn

Trong-Hieu Le*
Dept. of Computer Science
Ho Chi Minh City University of Information Technology
Ho Chi Minh City, Vietnam
25520545@gm.uit.edu.vn

***Abstract*—We address the detection and temporal localization of falls in continuous streams recorded by three ultra-wideband (UWB) radars. Background-residual and frame-difference maps are processed by a ResNet–LSTM and a CNN–TCN that predict, for every 0.1 s frame, the probability that a fall is in progress, starts or ends. The fused probabilities are decoded by a peak-anchored decoder that places event boundaries with the start and end outputs, and its detection threshold is adapted to each split without labels by matching the predicted event rate. Fusion weights and decoder are selected on out-of-fold (OOF) predictions of six subject-grouped folds, and the selection procedure is evaluated by nested cross-validation. The boundary outputs are the decisive component: on public validation they raise F1 at a temporal IoU (tIoU) of 0.5 to 0.83, against 0.63 when boundaries are searched near the run borders and 0.19 for the run borders themselves. Under nested evaluation the fused system reaches an event F1-mean of 0.841 over tIoU 0.1, 0.3 and 0.5, 0.02–0.03 above the best single branch under the same search space, mainly through fewer false positives; evaluating the selected configuration on its own selection data would have reported 0.880. On public validation the submitted system scores 0.906 (95% CI 0.856–0.951) with 0.49 false alarms per hour.**

***Index Terms*—Event detection, fall detection, model fusion, temporal localization, UWB radar**

I. Introduction

This section outlines the problem, the challenge setting and our contributions.

A. The Problem

Falls are a leading cause of injury for older adults living alone, and radar is attractive for home monitoring because it is contactless and does not record images [1], [2]. Most UWB fall studies classify pre-cut recordings as fall or non-fall [3], [4]. A deployed system instead observes a continuous signal in which falls are rare and last about two seconds. It must decide *when* each fall starts and ends, while rejecting walking, sitting, lying down and other daily activities that also produce strong reflections.

Three difficulties follow. First, static reflections dominate the raw radar return, so motion must be separated from the background. Second, event-level scores depend strongly on how frame probabilities are turned into events: a threshold slightly too low fragments one fall into several events, one slightly too high misses falls, and imprecise boundaries turn correct detections into errors at strict overlap thresholds. Third, the recordings come from few participants, so performance estimated on the data used to tune the decoder is optimistic [5], [6].

B. Challenge Context

The Fall Detection in Continuous UWB Radar Data Streams Challenge uses the LIARA UWB recordings [3], [4], [7]. It provides 30-minute streams composed of fall clips, daily-activity chunks and empty-room recordings captured by three radars, together with an official event-level scorer. Systems output scored intervals, which are matched one-to-one to annotated falls at temporal intersection-over-union (tIoU) thresholds $\theta \in \{0.1, 0.3, 0.5\}$; the ranking score is the mean of the three F1 values. We use the 13 training streams (359 falls) for six-fold subject-grouped cross-validation and the 5 public validation streams (119 falls, 67 scored) for development.

C. Key Contributions

A *peak-anchored decoder* that detects events on the fall probability and places their boundaries with dedicated start and end outputs, combined with label-free rate matching of the

*These authors contributed equally to this work and share first authorship.

detection threshold. An ablation shows that the boundary outputs account for most of the score at $\theta = 0.5$ (Section IV-B). Late fusion of a recurrent (ResNet–LSTM) and a convolutional (CNN–TCN-16s) branch, which raises the nested F1-mean by 0.02–0.03 over the best single branch, mainly by removing false positives; we show how this gain depends on the search space, the selection rule and the fold (Section IV-C). A selection protocol that separates the choice of fusion weights and decoder from the estimate of their performance. It quantifies the optimism of in-sample selection (3.9 points) and shows that a wider search space can lower the nested estimate of a single model (Sections III-F and IV-A).

II. Related Work

This section reviews UWB fall detection, fall detection on continuous streams, temporal localization and the evaluation of tuned systems.

A. UWB Radar Fall Detection

Amin et al. [1] review radar processing for in-home fall detection and argue for radar as a privacy-preserving alternative to cameras, and Hu et al. [2] extend this review to deep architectures. With millimetre-wave radar, Jin et al. [8] detect falls as anomalies in point-cloud sequences. The three-radar apartment used here was introduced by Maitre et al. [3], who classified simulated falls in fixed windows with a CNN–LSTM. The same group recognized daily activities with these radars [9] and compared deep and classical learners under subject- and position-wise splits [10]. Abudalfa and Bouchard applied pretrained vision models [11] and a hybrid vision transformer with a ResNet [4]. All of these works score window classification, so their accuracy and F1 values are not comparable with the event-level metric of this challenge.

B. Continuous Streams and Temporal Localization

Few fall studies evaluate continuous streams. Li et al. [12] classified sliding windows of FMCW radar data recorded as continuous activity sequences and reported window accuracy, and Aderinola et al. [13] tuned the decision threshold of a streaming accelerometer detector with a cost function that weighs missed falls against false alarms. Neither scores localized events by their temporal overlap with annotated falls.

Temporal convolutional networks were proposed for action segmentation and detection [14] and shown to be competitive with recurrent models for sequence modeling [15]. MS-TCN [16] adds a smoothing loss against over-segmentation, and ASRF [17] refines frame-wise predictions with a boundary branch. ActionFormer [18] and TriDet [19] predict boundaries directly and are scored at several tIoU thresholds. In radar sensing, Eat-Radar [20] segments intake gestures in continuous meal sessions, and Kobak et al. [21] segment transitional motions from MIMO radar. Our decoder follows the idea of separate action and boundary outputs, but uses them in a simple rule-based post-processing step.

TABLE I
Composition of the Development Splits

Split	Streams	Falls	Fall subj.	Scored	Flagged	Hours
Train (6-fold CV)	13	359	1–3, 5–7	186	173	6.18
Public validation	5	119	8, 9	67	52	2.05

Flagged falls act as ignore zones. ADL subjects: 4, 5, 6, 9, 10, 12 (train) and 0, 11 (validation). The OOF predictions cover 6.10 h because frames that mix two fold groups are excluded.

C. Evaluation of Tuned Systems

Tuning hyperparameters on the data used for estimation inflates performance [5], and the effect can be as large as the differences between methods [6]. We therefore report both the score of the selected configuration on the data used to select it and a nested estimate of the selection procedure.

III. Methodology

This section describes the dataset and splits, preprocessing, the two model branches, training, decoding, the selection protocol and the evaluation metrics.

A. Dataset and Splits

1) Source Recordings: The recordings were made with three XeThru X4M200 radars in an apartment of about 40 m^2; frames are aligned at 50 Hz with 368 range bins per radar [7]. Falls were simulated on a mat placed at three fixed positions [3], [7], and the package groups the fall clips by dataset (1–3, one per position) and by subject. Each fall subject contributes about 20 clips per position, with fall codes *Chute01–04* (French *chute*: fall). Daily activities come from a separate corpus of 15 activities recorded elsewhere in the apartment by different subjects [7]. A fall starts at the onset of the movement and ends when the person is stable on the floor; its bounds carry an uncertainty of ± 0.5–1 s because the radar clocks use integer seconds [7]. Annotated falls last 1.7 s at the median (0.5–5.0 s).

2) Stream Composition: The challenge generator packs each split into streams of at most 30 minutes [7]. A fall clip keeps 3–8 s of context before the annotated fall and 5–10 s after it; clips in the streams last 7.3–18.6 s. Each daily-activity (ADL) file is cut into chunks of 8–20 s that are circularly shifted by a random offset, so the chunks actually used last 0.5–20 s. ADL files are cut three times for training and twice for validation and test, and chunks are added until the split reaches about 57 falls per hour. Recordings labelled *Autre* (other activity) and *Empty* are inserted whole, and segments are joined with a 0.2 s crossfade.

3) Flagged Falls: Falls with low annotation quality (residual below a gate, mainly in Dataset 3) or a review flag remain in the streams but act as ignore zones in the official test rule [7]. This affects 225 of the 478 development falls, including 104 of the 120 Dataset 3 falls in training and all 39 in validation. Table I summarizes the two development splits; no source recording appears in both.

4) Cross-Validation Folds: Each of the six folds holds out one fall subject k (all three positions), the *Autre*/*Empty*

recordings of the same subject and one ADL subject m, with $(k, m) \in \{(1,4),(2,5),(3,6),(5,9),(6,10),(7,12)\}$. We denote the folds f1, f2, f3, f5, f6 and f7 by their fall subject. Only the stream files and the stream composition table are used; no source recording is accessed, as required by the challenge rules [7].

B. Preprocessing

1) Background Estimation and Feature Extraction: Let x_t denote the channel impulse response of one radar at 50 Hz over 368 range bins. A causal exponential moving average (EMA) estimates the static background,

$$b_t = \alpha b_{t-1} + (1-\alpha)x_t, \qquad \alpha = \exp\left(-\frac{1}{50\tau}\right), \quad \tau = 3\,\mathrm{s}, \tag{1}$$

using the time constant of the official energy baseline [7]. Two magnitude maps are then constructed: the background residual $r_t = |x_t - b_t|$, which preserves moving targets, and the frame difference $d_t = |x_t - x_{t-1}|$, which emphasizes rapid changes such as the descent phase of a fall.

2) Spatial and Temporal Downsampling: Both magnitude maps are average-pooled over four adjacent range bins, reducing the range dimension from 368 to 92 bins. Temporal average pooling is performed over blocks of five consecutive samples, reducing the sampling rate from 50 to 10 Hz. Each resulting frame is timestamped at the centre of its corresponding block.

3) Stream-Level Feature Construction: Feature extraction is performed once over each entire stream so that the background filter runs continuously across segment junctions. Fold-specific views are then extracted from these precomputed features. The residual and difference maps from the three radars are combined to form six input channels.

4) Normalization: Each channel is standardized using its mean and standard deviation fitted exclusively on the training portion of the corresponding fold. The standardized values are subsequently clipped to the range $[-8, 8]$.

C. Model Development

Fig. 1 shows the two branches. Both map a six-channel window to three frame-wise logits: a fall is in progress (*action*), a fall starts, and a fall ends. This action–boundary decomposition follows boundary-aware localization methods [17], [19].

1) ResNet–LSTM: The branch takes an 80-frame (8 s) window. A 2D residual network [22] with a 5×5 stem and four stages of two 3×3 basic blocks (16, 32, 48 and 64 channels, batch normalization, ReLU, spatial dropout 0.1) strides along range only. Its kernels also slide over time, with a temporal receptive field of 37 frames (3.7 s). Averaging over range gives a $T \times 64$ sequence, which a two-layer bidirectional LSTM [23] with 64 hidden units per direction and dropout 0.2 turns into a $T \times 128$ sequence; a linear layer maps it to the three logits. The branch has 431,139 parameters.

2) CNN–TCN-16s: The branch takes a 160-frame (16 s) window. A range CNN with 1×7 and 1×5 kernels (24 and 48 channels, group normalization, GELU) encodes each frame separately, and averaging over range gives a $T \times 48$ sequence. A temporal convolutional network [14], [15] of four residual blocks (64, 64, 96 and 96 channels, kernel 5, dilations 1, 2, 4 and 8, dropout 0.2) has a receptive field of $1 + 2 \cdot 4 \cdot (1 + 2 + 4 + 8) = 121$ frames (12.1 s); three 1×1 convolutions produce the logits after dropout 0.25. The branch has 264,331 parameters.

3) CNN–TCN-8s Candidate: A shorter configuration with an 80-frame window, three TCN blocks (dilations 1, 2 and 4; receptive field 5.7 s; 171,595 parameters) and no augmentation served as a third fusion candidate. Since CNN–TCN-16s changes both the context length and the augmentation, the two effects cannot be separated.

D. Training

1) Targets and Loss: For an annotated fall $[t_s, t_e]$, the action target is 1 inside the interval and decreases linearly to 0 within 0.3 s outside it; start and end targets are Gaussians centred on t_s and t_e with $\sigma = 0.2$ s. The loss is binary cross-entropy with logits, with positive weights $(3, 5, 5)$ and output weights $(1, 0.5, 0.5)$ for (action, start, end), normalized by the sum of frame weights. Frames within 2 s of a fall have weight 0 if the fall is marked *ignore* (including falls truncated at a view border) and 0.35 if it is flagged; flagged falls are thus still trained as positives although they are not scored.

2) Window Sampling: Candidate windows are placed with a hop of 20 frames. Each fall contributes at most three windows that overlap it, with a random shift of up to ± 10 frames that keeps the overlap, and negative windows are drawn at three times the number of positives. Each epoch draws 110×32 windows with replacement.

3) Augmentation (CNN–TCN-16s only): With probability 0.5 a window is multiplied, per radar, by a gain drawn from $U(0.8, 1.25)$ before standardization, with the same factor for the residual and difference channels of that radar. After standardization, two time masks of 0–5 frames and two range masks of 0–8 bins are set to zero.

4) Optimization: Members are trained with AdamW, batch size 32, gradient clipping at 1.0 and a fixed number of epochs without early stopping: 20 epochs with learning rate $3 \cdot 10^{-4}$ and weight decay $2 \cdot 10^{-4}$ for the CNN–TCN branches, and 30 epochs with learning rate 10^{-3} and weight decay 10^{-2} for the ResNet–LSTM. The learning rate is halved by a plateau scheduler (patience 1) that monitors the loss on the held-out fold. The final weights of a member are the average of the three epochs centred on the epoch with the best smoothed event F1 on the held-out fold, measured with a fixed decoder ($high = 0.6$, $low = 0.2$). One member is trained per fold and seed (2027, 2028), i.e., 12 members per branch. Training took about 3.2 min per CNN–TCN-16s member and 4.4 min per ResNet–LSTM member on an NVIDIA L4 Tensor Core GPU.

E. Inference and Event Decoding

1) Sliding-Window Inference: Windows are applied with a hop of 2 frames (the last window is edge-padded), and the

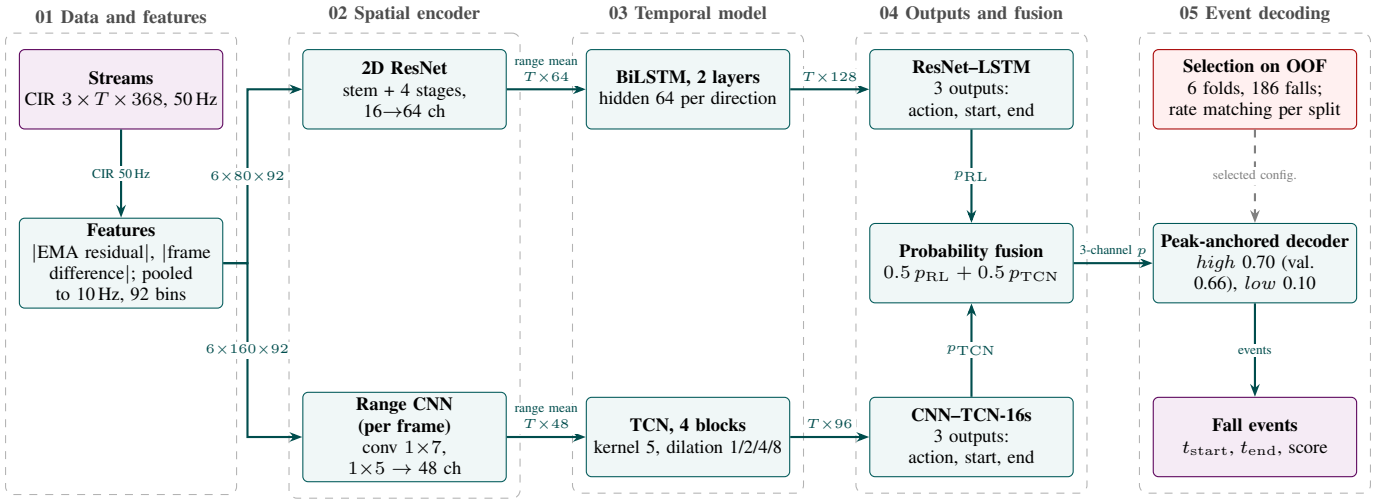


Fig. 1. Architecture of the submitted system. (01) Stream features and the window fed to each branch (channels × frames × range bins). (02) Spatial encoders: the range CNN processes each frame separately, whereas the 2D ResNet also convolves over time. (03) Temporal models with their output size per frame. (04) Three frame-wise outputs per branch and probability fusion, applied to all three channels. (05) Peak-anchored decoder with the configuration selected on the out-of-fold predictions; its *high* threshold is re-adapted to each split by rate matching. The CNN–TCN-8s candidate received weight 0 and is not shown.

probability of a frame is the mean over all windows covering it. Each member uses the normalizer of its own fold. For validation and test, the two seeds of each fold are averaged within a branch, the branches are combined with the fusion weights $p(t) = \sum_m w_m\, p_m(t)$ on all three channels, and the result is averaged over the six folds. No model is retrained on all training data.

2) Peak-Anchored Decoder: The action channel is smoothed with a 5-frame moving average and the start and end channels with a 3-frame moving average. A candidate event is a run $[a, b)$ of frames whose smoothed action probability is at least *low*; runs whose maximum is below *high* are discarded. Within a run, the peak frame c is the action maximum, the start is the frame with the highest start probability in $[a-10, c]$ and the end is the frame with the highest end probability in $[c, b+9]$. If the resulting duration falls outside $[0.5, 8]$ s, the run borders are used instead. Events closer than 0.5 s are merged if the merged event does not exceed 8 s, and the event score is the 90th percentile of the smoothed action probability within the run.

A *border-anchored* variant searches the start in $[a-10, a+10]$ and the end in $[b-11, b+9]$, i.e., only around the run borders. It is part of the search space and is used for checkpoint selection.

3) Search Space: Fusion weights for the three candidates (ResNet–LSTM, CNN–TCN-8s, CNN–TCN-16s) range over the 15 combinations on a 0.25 grid that sum to one. For each, the two decoder variants are combined with 43 pairs of thresholds (*high* from 0.40 to 0.90 in steps of 0.05, $low \in \{0.1, 0.2, 0.3, 0.4\}$, $low < high$), giving 1,290 configurations; minimum duration, maximum duration and merge gap are fixed at 0.5, 8 and 0.5 s.

4) Label-Free Rate Matching: The configuration is selected on OOF predictions, in which every frame is predicted by the two members of one fold. On validation and test, every frame is the average of twelve members, whose peaks are lower and broader, so the same *high* yields fewer events. On each split we therefore count the events per hour predicted by each of the six fold models separately (45.3 per hour on validation) and choose $high \in [0.30, 0.96]$ (step 0.01, above *low*) such that the fold-averaged ensemble predicts the same rate. On validation this gives $high = 0.66$ instead of 0.70. The procedure uses no labels but is transductive, and it assumes a similar event rate across splits, which the stream generator ensures by targeting the same fall rate for all splits [7]. It was adopted after an in-sample simulation on the OOF predictions, in which a decoder selected on one seed was transferred to the two-seed average: matching the rate raised F1-mean from 0.870 to 0.881 (seed 2027) and from 0.861 to 0.873 (seed 2028).

F. Selection and Estimation Protocol

A *member* is one trained network (one fold, one seed), a *branch* is the set of members of one architecture, and a *configuration* is a choice of fusion weights, decoder variant and thresholds. The *OOF predictions* are the held-out predictions of all six folds, pooled into one set (186 scored falls, 6.10 h).

The submitted configuration is *selected* on the pooled OOF predictions of all six folds by one of three rules: *argmax* F1-mean; *plateau-thr*, which scores each configuration by the mean over its 3×3 neighbourhood in (*high*, *low*); and *plateau-all*, which also averages over weight vectors that differ by at most 0.25 in each component, with the same decoder variant. Ties are broken by the raw score and then by the false-alarm rate. Plateau rules prefer configurations whose neighbours also score well, which reduces the chance of selecting a lucky threshold.

The selection procedure is *estimated* by nested cross-validation (Fig. 2): for each fold, a configuration is selected

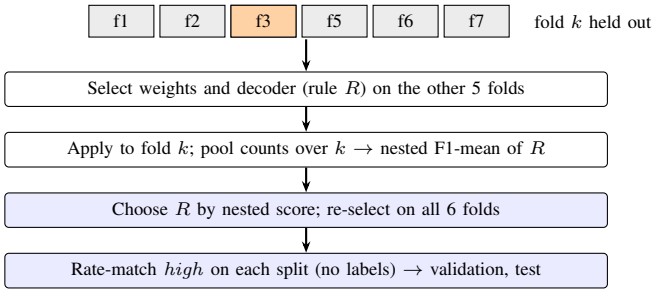


Fig. 2. Selection and estimation protocol over the six subject-grouped folds. The held-out fold (orange) never influences the configuration applied to it. Checkpoint selection inside each member uses the held-out fold and is outside this loop (Section IV-F).

on the other five folds with the same rule and applied to the held-out fold, and the counts of the six held-out folds are pooled into a *nested* F1-mean. The rule itself is chosen by its nested score. The *in-sample* score evaluates the finally selected configuration on the OOF predictions used to select it.

G. Evaluation Metrics

All scores use the official scorer in the mode of the official test, in which flagged falls act as ignore zones [7]. Predictions are sorted by score and each is matched to the unmatched, non-ignored fall with the highest tIoU, provided $\mathrm{tIoU} \geq \theta$. An unmatched prediction is a false positive (FP) unless it overlaps an ignored fall widened by ± 2 s, in which case it is discarded. Precision, recall and F1 are computed from true positives (TP), FP and false negatives (FN) pooled over all streams, and the ranking score is

$$\text{F1-mean} = \tfrac{1}{3}\left(\mathrm{F1}_{0.1} + \mathrm{F1}_{0.3} + \mathrm{F1}_{0.5}\right). \qquad (2)$$

Ties are broken by the median absolute start error at $\theta = 0.3$ and then by the false alarms per hour (FA/h: FP at $\theta = 0.3$ divided by the stream duration). Unless stated otherwise, scores refer to pooled counts and TP/FP/FN are given at $\theta = 0.3$.

IV. Results and Discussion

This section reports the estimates of the submitted system, the effect of the decoder and of fusion, validation results and an error analysis, and then discusses what limits and what supports these results.

A. Overview of the Estimates

Table II lists the estimates of the submitted system and what each one tunes on the data it is computed on. The in-sample OOF score (0.880) exceeds the nested estimate (0.841) by 3.9 points, which measures the optimism of selecting and evaluating on the same predictions. The public validation score (0.906) is higher still, but rests on 67 scored falls from two subjects.

TABLE II
Estimates of the Submitted System

Estimate	Tuned on the evaluated data	TP/FP/FN	F1-mean
OOF, in-sample	configuration, rule, checkpoints	163/11/23	0.880
OOF, nested	rule, checkpoints	154/16/32	0.841
Public validation	$high$ by rate matching (no labels)	60/1/7	0.906
Validation, $high = 0.70$	none	59/1/8	0.898
Hidden test	$high$ by rate matching	[–]	[–]

"Checkpoints": member checkpoints and learning-rate schedules used the held-out fold. FA/h: 1.80 in-sample OOF (6.10 h), 0.49 on validation (2.05 h). On the test split $high$ is rate-matched again.

TABLE III
Decoder Ablation on Public Validation (Fused Probabilities)

Event boundaries	$\mathrm{F1}_{0.1}$	$\mathrm{F1}_{0.3}$	$\mathrm{F1}_{0.5}$	F1-mean	TP/FP/FN
Action run borders	0.945	0.819	0.189	0.651	52/8/15
Border-anchored	0.953	0.891	0.625	0.823	57/4/10
Peak-anchored	0.953	0.938	0.828	**0.906**	60/1/7

Submitted fusion weights, $high = 0.66$, $low = 0.10$. Median absolute start / end error at $\theta = 0.3$: 1.38 / 1.06 s, 0.44 / 0.27 s and 0.18 / 0.25 s.

B. Effect of the Event Decoder

Table III decodes the same fused validation probabilities in three ways. Using the borders of the action run as event boundaries finds most falls at $\theta = 0.1$ but fails at strict overlap ($\mathrm{F1}_{0.5} = 0.19$, median start error 1.38 s), because runs above $low = 0.1$ extend well beyond the fall. Searching the start and end outputs near the run borders already raises $\mathrm{F1}_{0.5}$ to 0.63; anchoring the search at the action peak raises it to 0.83 and reduces the median start error to 0.18 s. Since low was selected for the peak-anchored decoder, the run-border row is a lower bound for an action-only decoder; the comparison between the two boundary searches is like for like.

Table IV repeats the comparison under nested selection on the OOF predictions, restricting the search to one decoder variant. The peak-anchored decoder raises $\mathrm{F1}_{0.5}$ for every system that contains a CNN–TCN, by 0.02–0.06, and gives the best nested F1-mean of the fused search (0.845). Searching over both variants does not help: for CNN–TCN-16s it lowers the nested estimate from 0.820–0.822 to 0.813, because the wider search selects less stable configurations on five folds. The submitted configuration was obtained with the search over both variants.

C. Effect of Fusion and of the Selection Rule

Table V evaluates every single branch and every fusion under the three selection rules. With the search used for submission, the fused search reaches 0.841 against 0.813 for the best single branch (CNN–TCN-16s), a gain of 0.028; under the same single-decoder searches of Table IV the gain is 0.018 (border-anchored) and 0.023 (peak-anchored). At $\theta = 0.3$ the fusion keeps almost the same number of detections (154 vs.

TABLE IV
NESTED OOF F1-MEAN ($F1_{0.5}$) BY DECODER SEARCH

System	Border-anch.	Peak-anch.	Both (used)
ResNet–LSTM	0.793 (0.687)	0.793 (0.698)	0.793 (0.698)
CNN–TCN-8s	0.809 (0.712)	0.799 (0.752)	0.809 (0.712)
CNN–TCN-16s	0.820 (0.734)	0.822 (0.754)	0.813 (0.723)
ResNet–LSTM + CNN–TCN-8s	0.813 (0.698)	0.821 (0.755)	0.814 (0.751)
Search over all three	0.838 (0.748)	0.845 (0.781)	0.841 (0.770)

Plateau-all rule, 186 scored falls. "Search over all three" is the search that produced the submitted system (ResNet–LSTM + CNN–TCN-16s, weights 0.5/0.5).

TABLE V
NESTED OOF F1-MEAN BY SYSTEM AND SELECTION RULE

System	argmax	plateau-thr	plateau-all
ResNet–LSTM	0.786	0.793	0.793
CNN–TCN-8s	0.804	0.809	0.809
CNN–TCN-16s	0.808	0.813	0.813
ResNet–LSTM + CNN–TCN-8s	0.841	0.824	0.814
ResNet–LSTM + CNN–TCN-16s	0.830	0.837	0.849
CNN–TCN-8s + CNN–TCN-16s	0.806	0.801	0.805
Search over all three	0.827	0.825	0.841

Search over both decoder variants, 186 scored falls. The submitted system is the search over all three with plateau-all; it assigned weight 0 to CNN–TCN-8s, so its weights coincide with ResNet–LSTM + CNN–TCN-16s, whose own search averages a smaller weight neighbourhood under plateau-all.

TABLE VI
NESTED RESULTS PER HELD-OUT FOLD

Fold	Falls	CNN–TCN-16s		Submitted fusion	
		F1-mean	TP/FP/FN	F1-mean	TP/FP/FN
f1	27	0.723	19/1/8	0.776	21/1/6
f2	19	0.982	19/0/0	0.965	18/1/1
f3	50	0.805	36/1/14	0.784	34/1/16
f5	29	0.867	29/2/0	0.893	27/3/2
f6	30	0.761	28/13/2	0.844	28/6/2
f7	31	0.796	25/6/6	0.842	26/4/5
Mean of folds	–	0.822	–	0.850	–
Pooled	186	0.813	156/23/30	0.841	154/16/32

Plateau-all rule, search over both decoder variants; each fold is scored with the configuration selected on the other five folds.

TABLE VII
SUBMITTED SYSTEM ON PUBLIC VALIDATION (67 SCORED FALLS)

θ	TP	FP	FN	Precision	Recall	F1
0.1	61	0	6	1.000	0.910	0.953
0.3	60	1	7	0.984	0.896	0.938
0.5	53	8	14	0.869	0.791	0.828
0.7*	40	21	27	0.656	0.597	0.625
F1-mean (0.1/0.3/0.5)						**0.906**

*Not part of the ranking score. Mean tIoU of matches at $\theta = 0.3$: 0.735. 95% bootstrap intervals over composition segments: $F1_{0.1}$ $[0.911, 0.985]$, $F1_{0.3}$ $[0.884, 0.980]$, $F1_{0.5}$ $[0.737, 0.909]$, F1-mean $[0.856, 0.951]$.

156 TP) and reduces FP from 23 to 16. Every fusion that contains the ResNet–LSTM scores at least 0.814 under every rule, whereas the fusion of the two convolutional branches does not improve on CNN–TCN-16s alone (at most 0.806). The size of the gain depends on the rule: ResNet–LSTM + CNN–TCN-8s reaches 0.841 with argmax (156/14/30) but 0.814 with plateau-all. A bootstrap over the six folds (5,000 resamples, therefore coarse) gives a 95% interval of $[-0.003, +0.059]$ for the gain over CNN–TCN-16s and $[-0.038, +0.036]$ for the difference from ResNet–LSTM + CNN–TCN-8s with argmax. The data therefore support a gain from adding the ResNet–LSTM to a CNN–TCN, but not a specific advantage of CNN–TCN-16s over CNN–TCN-8s in the fusion.

Table VI breaks the nested comparison down by held-out fold. Fold difficulty varies widely: f2 is almost solved, whereas f1 and f3 are hard for both systems. The reduction of FP is concentrated in f6, where the configuration selected for CNN–TCN-16s on the other five folds produced 13 FP against 6 for the fusion. In f3 the fusion misses two more falls than CNN–TCN-16s. The misses of CNN–TCN-16s in f3 depend mostly on the threshold: with the fixed checkpoint-selection decoder ($high = 0.6$) it detects 43 of the 50 falls, with 5 FP, against 36 with 1 FP for the threshold selected on its pooled OOF predictions ($high = 0.9$). The configurations selected inside the nested loop also differ: without f3 the search chose weights $(0, 0.25, 0.75)$ for (ResNet–LSTM, CNN–TCN-8s, CNN–TCN-16s) with thresholds 0.9/0.2, and without f6 it chose the border-anchored decoder. The final selection on all six folds gave weights $(0.5, 0, 0.5)$, the peak-anchored decoder, $high = 0.7$ and $low = 0.1$.

D. Public Validation

Table VII reports the submitted system on public validation. At $\theta = 0.3$ it detects 60 of the 67 scored falls with one FP (precision 0.984, recall 0.896), i.e., 0.49 FA/h over 2.05 h. That FP is a late detection of a scored fall (tIoU 0.23, matched at $\theta = 0.1$), so no daily activity triggered a detection on validation; 32 further predictions overlap flagged falls and are discarded by the scorer. The median absolute start and end errors are 0.18 s and 0.25 s, and 75% and 92% of the start times lie within 0.5 s and 1 s. A bootstrap over the 588 composition segments of the validation streams (10,000 resamples, matching fixed) gives a 95% interval of $[0.856, 0.951]$ for the F1-mean. Leaving out one stream changes the F1-mean by -0.015 to $+0.030$.

Table VIII decomposes the validation score. With the configurations each branch selects on its own OOF predictions, the ResNet–LSTM and the CNN–TCN-16s reach 0.797 and 0.847. Applying the fusion decoder to each branch raises them to 0.850 and 0.879, mostly through more true positives; fusion then removes false positives (from 5 to 1 compared with CNN–TCN-16s alone) and reaches 0.898 with the OOF threshold and 0.906 after rate matching. Averaging the six fold models adds about 0.014 over a single fold model (0.848–0.906, mean 0.883, same decoder). All systems are far above the official baselines, which were tuned on validation itself.

Fig. 3 shows why the threshold needs care. On the OOF predictions, F1-mean peaks at $high = 0.70$ (0.880) and falls

TABLE VIII
ABLATION ON PUBLIC VALIDATION

System	Decoder ($high$/low)	TP/FP/FN	F1-mean
Energy baseline	tuned on validation	–	0.315
Logistic baseline	tuned on validation	–	0.258
ResNet–LSTM	own OOF: peak 0.9/0.1	49/2/18	0.797
CNN–TCN-16s	own OOF: border 0.9/0.2	54/1/13	0.847
ResNet–LSTM	fusion: peak 0.7/0.1	57/3/10	0.850
CNN–TCN-16s	fusion: peak 0.7/0.1	60/5/7	0.879
Fusion	peak 0.70/0.1	59/1/8	0.898
Fusion (submitted)	peak 0.66/0.1 (rate-m.)	60/1/7	**0.906**

Single branches average their 12 members. "peak" and "border": peak- and border-anchored decoders. Baselines from the challenge documentation [7].

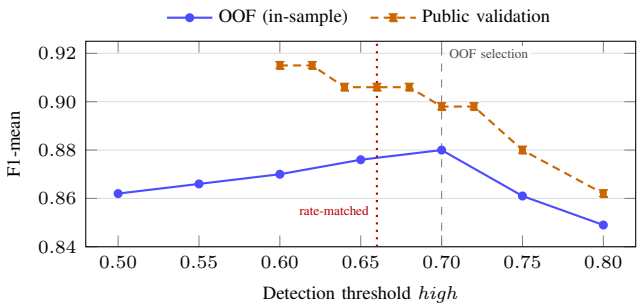


Fig. 3. F1-mean of the submitted configuration as a function of the $high$ threshold ($low = 0.1$). The dashed grey line marks the value selected on the OOF predictions and the dotted red line the value obtained on validation by rate matching. The validation curve is shown for sensitivity only and was not used for selection.

steeply above it (0.861 at 0.75), so the selected value lies at the upper edge of the good region. On validation the curve is flatter and its best values lie lower (0.915 at 0.60–0.62); rate matching, which uses no labels, moved the threshold in that direction (0.66, 0.906).

E. Error Analysis

Table IX breaks down the validation detections by fall code and dataset. Chute01–03 are detected in 52 of 53 cases, whereas Chute04 is detected in 8 of 14, and six of the seven misses are Chute04 falls of Dataset 1. The remaining errors therefore concentrate on one fall code at one position rather than spreading over the data. Annotated durations do not explain the gap (median 2.0 s for Chute04 versus 1.7–1.9 s for the other codes). We could not verify the mapping between the challenge fall codes and the fall types described in the source study, so we do not attribute the lower detection rate for Chute04 to a specific fall type. No Dataset 3 fall is scored on validation, since all are flagged.

Fig. 4 shows an excerpt of one validation stream. The scored Chute01 fall at 38 s produces a sharp action peak close to 1, and the start and end outputs peak near its boundaries, so the decoded event matches it. The flagged Chute02 fall at 28 s produces an equally clear peak, and its prediction is

TABLE IX
VALIDATION DETECTIONS BY FALL CODE AND DATASET

Fall code	Dataset 1	Dataset 2	Total
Chute01	4/5	9/9	13/14
Chute02	9/9	10/10	19/19
Chute03	10/10	10/10	20/20
Chute04	**3/9**	5/5	**8/14**
Total	26/33	34/34	60/67

Detected / scored falls at $\theta = 0.3$ for the submitted system. Dataset 3 has no scored fall on validation (39 of 39 flagged).

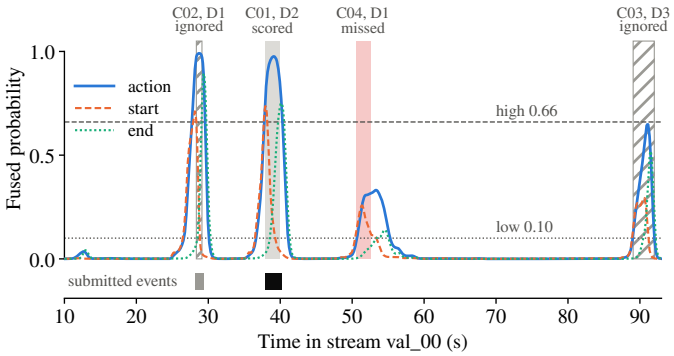


Fig. 4. Excerpt of validation stream val_00 (10–93 s). Lines: fused action, start and end probabilities after the decoder smoothing; dashed and dotted horizontals: $high = 0.66$ and $low = 0.10$. Shaded intervals: scored falls (red: the missed Chute04 fall); hatched: flagged falls (ignore zones). Bars below zero: submitted events (black: true positive; grey: discarded in an ignore zone). Labels: C0k = Chute0k, Dk = Dataset k.

discarded by the ignore rule. The Chute04 fall of Dataset 1 at 51 s produces a weak, delayed response: the action probability rises shortly after the annotated start and peaks at 0.33 about 0.8 s after the annotated end, above low but below $high$, so the fall is missed although the model reacts to it. The flagged Dataset 3 fall at 89 s peaks just below the threshold (0.65). At $\theta \leq 0.3$ the remaining validation errors are thus mostly misses (6 FN against 0 FP at $\theta = 0.1$), whereas boundary errors appear at stricter overlap: TP decrease from 60 to 53 between $\theta = 0.3$ and 0.5 and to 40 at 0.7.

F. Discussion

What drives the score. The decoder contributes more than fusion. Placing boundaries with the start and end outputs is what makes the detections count at $\theta = 0.5$ (Table III), while fusion adds 0.02–0.03 to the nested F1-mean, mostly by removing false positives in one fold. Fusing the two convolutional branches did not help, which suggests that the benefit comes from combining different architectures rather than from averaging more models; we did not inspect individual errors to confirm this mechanism.

Strict overlap thresholds. The annotated bounds are uncertain by ± 0.5–1 s, while a typical fall lasts 1.7 s. For a prediction of the correct length shifted by s seconds, the tIoU is $(1.7 - s)/(1.7 + s)$, which falls below 0.5 for $s > 0.57$ s and below 0.7 for $s > 0.3$ s. The ranking thresholds up to

0.5 are therefore compatible with the label precision, whereas $\theta = 0.7$ asks for boundary errors smaller than the annotation uncertainty; the drop of F1 from 0.828 to 0.625 between these thresholds should be read with this in mind.

Possible shortcut. The challenge documentation reports that a pre-fall window ending 0.5 s before the annotated start already separates falls from other activities (AUC 1.000 against ADL and 0.839 against other or empty recordings), because falls were simulated on mats at fixed positions while activities were recorded elsewhere. Background removal with time constants of 2 s and 8 s leaves this AUC at 0.923 and 0.960 [7], so our 3 s filter is unlikely to remove the cue. The 16 s window of CNN–TCN-16s was chosen to see the phase preceding a fall and may exploit it more than the 8 s branches. We did not run a control, such as masking the pre-fall context, and the scores should be read with this possibility in mind. Segment junctions are a weaker concern, since fall clips are cropped with random margins so that a junction does not mark a fall [7].

Reliability of the estimates. The nested estimate separates the selection of the configuration from its evaluation, but not every choice. Member checkpoints and learning-rate schedules used the held-out fold; the selection rule and the use of rate matching were chosen on the OOF predictions; the design of CNN–TCN-16s (longer context, augmentation) followed an error analysis of earlier OOF results; and the public validation split was scored during development. A replacement criterion fixed before training this system (nested F1-mean ≥ 0.847) was not met Nevertheless, we selected this system because it improved both the nested OOF F1-mean (0.831 to 0.841) and the public-validation F1-mean (0.853 to 0.906) over the previous ensemble, while reducing validation false alarms from 0.98 to 0.49 per hour. Although the configuration was fixed before validation evaluation, this final selection overrode the pre-specified replacement criterion and relied partly on public-validation results. The nested 0.841 is therefore an upper estimate of a fully nested procedure, and the validation score is a development score: its 95% interval spans almost ten points, and one additional FP would change FA/h by 0.49 and $\mathrm{F1}_{0.3}$ by 0.007.

Scope. Only the background filter is causal. The bidirectional LSTM, the symmetric padding of the TCN, the averaging of overlapping windows, the symmetric smoothing and the boundary search, which looks up to one second beyond a run, all use future frames, so the system is an offline localizer and its start errors are not alarm latencies. Every fold contains all three positions, so generalization to new positions [10] was not tested, and the participants of the source recordings are not older adults [3]. Because flagged falls are ignore zones and nearby predictions are discarded, FA/h understates the false-alarm rate in unfiltered use.

V. Conclusion

This study addressed event-level fall localization in streams recorded by three UWB radars. Background-residual and frame-difference maps were processed by a ResNet–LSTM and a CNN–TCN, whose fused fall, start and end probabilities were decoded by a peak-anchored decoder with a label-free, rate-matched threshold. Weights and decoder were selected on out-of-fold predictions, and the selection procedure was evaluated by nested cross-validation.

Three findings stand out. First, dedicated boundary outputs are the most important component for the ranking metric: on validation they raise F1 at $\theta = 0.5$ from 0.63 (border-anchored search) to 0.83. Second, fusing a recurrent and a convolutional branch adds 0.02–0.03 to the nested F1-mean, mainly by removing false positives, and the size of this gain depends on the search space, the selection rule and the fold. Third, evaluating the selected configuration on its own selection data would have overstated performance by 3.9 points. Results on the hidden test split were not available at the time of writing. The public-validation score reflects development-stage performance, while the nested estimate reduces configuration-selection bias but remains subject to the limitations discussed in Section IV-F.

The remaining validation misses concentrate on Chute04 falls at one position, and a pre-fall shortcut has not been ruled out. Future work should therefore (i) run controls that mask the pre-fall context and test on held-out positions; (ii) study the missed Chute04 falls, whose weak responses suggest that a boundary-aware decoder with a lower, event-specific threshold could recover some of them; (iii) retrain members with inner-fold checkpoint selection to obtain a fully nested estimate; (iv) separate the effects of context length and augmentation; and (v) replace the rule-based boundary search with learned boundary regression [17], [19] and a causal temporal model so that alarm latency can be measured.

Code Availability

The training and inference notebooks and the trained members will be provided to the organizers, as required for top-ranked teams [7]. The code is available at https://github.com/khoile2550109-cyber/ABC-Challenge-2027.

Acknowledgment

The data were provided by the LIARA laboratory, Université du Québec à Chicoutimi, under CC BY-NC 4.0.

References

[1] M. G. Amin, Y. D. Zhang, F. Ahmad, and K. C. D. Ho, "Radar signal processing for elderly fall detection: The future for in-home monitoring," *IEEE Signal Processing Magazine*, vol. 33, no. 2, pp. 71–80, Mar. 2016.

[2] S. Hu, S. Cao, N. Toosizadeh, J. Barton, M. G. Hector, and M. J. Fain, "Radar-based fall detection: A survey," *IEEE Robotics & Automation Magazine*, vol. 31, no. 3, pp. 170–185, 2024.

[3] J. Maitre, K. Bouchard, and S. Gaboury, "Fall detection with UWB radars and CNN-LSTM architecture," *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 4, pp. 1273–1283, Apr. 2021.

[4] S. Abudalfa and K. Bouchard, "A hybrid vision transformer and residual neural network model for fall detection using UWB radars," *Applied Intelligence*, vol. 55, no. 3, Feb. 2025, Art. no. 222.

[5] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, 2006, Art. no. 91.

[6] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010.

[7] ABC2027 UWB Challenge organizers, "ABC2027 UWB fall challenge: training and validation package (abc2027_uwb_train_v1) and documentation," https://sites.google.com/view/abc2027-uwbfalltrack-challenge, 2026, accessed Nov. 2026.

[8] F. Jin, A. Sengupta, and S. Cao, "mmFall: Fall detection using 4-D mmWave radar and a hybrid variational RNN AutoEncoder," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 2, pp. 1245–1257, Apr. 2022.

[9] J. Maitre, K. Bouchard, C. Bertuglia, and S. Gaboury, "Recognizing activities of daily living from UWB radars and deep learning," *Expert Systems with Applications*, vol. 164, 2021, Art. no. 113994.

[10] T. Imbeault-Nepton, J. Maître, K. Bouchard, and S. Gaboury, "Fall detection from UWB radars: A comparative analysis of deep learning and classical machine learning techniques," in *Proc. ACM Int. Conf. Information Technology for Social Good (GoodIT '23)*, Lisbon, Portugal, 2023, pp. 197–204.

[11] S. Abudalfa and K. Bouchard, "Transfer learning using computer vision models for fall detection from UWB radars," *Transportation Research Procedia*, vol. 84, pp. 19–26, 2025.

[12] H. Li, A. Shrestha, H. Heidari, J. Le Kernec, and F. Fioranelli, "Activities recognition and fall detection in continuous data streams using radar sensor," in *Proc. IEEE MTT-S Int. Microwave Biomedical Conf. (IMBioC)*, Nanjing, China, May 2019, pp. 1–4.

[13] T. B. Aderinola, L. Palmerini, I. D'Ascanio, L. Chiari, J. Klenk, C. Becker, B. Caulfield, and G. Ifrim, "Watch your step: A cost-sensitive framework for accelerometer-based fall detection in real-world streaming scenarios," arXiv:2509.11789, 2025.

[14] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1003–1012.

[15] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018.

[16] Y. Abu Farha and J. Gall, "MS-TCN: Multi-stage temporal convolutional network for action segmentation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3570–3579.

[17] Y. Ishikawa, S. Kasai, Y. Aoki, and H. Kataoka, "Alleviating over-segmentation errors by detecting action boundaries," in *Proc. IEEE Winter Conf. Applications of Computer Vision (WACV)*, 2021, pp. 2321–2330.

[18] C.-L. Zhang, J. Wu, and Y. Li, "ActionFormer: Localizing moments of actions with transformers," in *Proc. European Conf. Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, vol. 13664. Cham: Springer, 2022, pp. 492–510.

[19] D. Shi, Y. Zhong, Q. Cao, L. Ma, J. Li, and D. Tao, "TriDet: Temporal action detection with relative boundary modeling," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 18 857–18 866.

[20] C. Wang, T. S. Kumar, W. De Raedt, G. Camps, H. Hallez, and B. Vanrumste, "Eat-Radar: Continuous fine-grained intake gesture detection using FMCW radar and 3D temporal convolutional network with attention," *IEEE Journal of Biomedical and Health Informatics*, vol. 28, no. 2, pp. 1000–1011, Feb. 2024.

[21] J. Kobak, B. J. Richman, L. Washington, and S. A. Hamza, "Continuous human activity recognition using a MIMO radar for transitional motion analysis," in *Proc. SPIE 12522, Big Data V: Learning, Analytics, and Applications*, Orlando, FL, USA, 2023, Art. no. 125220D.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[23] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.